7

PART 2: FRAMEWORK

Skills: reusable implementation knowledge

How do you encode a team's implementation conventions once instead of repeating them in every prompt?

A developer at Riverside Clinics sits down to ask an AI assistant to implement the Schedule Appointment use case from Chapter 3, against the domain model from Chapter 4 and the architecture rules from Chapter 6. The request should be one sentence. Instead she opens a blank file, because she remembers what happened the last time she sent the short version: the assistant picked a testing library nobody on the team uses, logged nothing useful when a booking failed, and wrapped every error in a generic exception the rest of the codebase doesn't recognize.

So she starts typing everything the short version left out. Use JUnit 5 and Testcontainers for anything that touches the database. Log every appointment state change with its ID and the actor who caused it. Validate incoming requests with Jakarta Validation annotations, not hand-written checks. Wrap repository failures in a domain-specific exception, and return RFC 7807 problem-details responses for anything the API rejects. Use constructor injection, never field injection. Ten minutes in, the request is four paragraphs long, and she still isn't sure she's remembered everything the last three code reviews corrected.

CHAPTER PROMISE

By the end of this chapter, you can tell an implementation habit apart from a structural constraint, write that habit down once as a skill, and hand an AI assistant a request short enough to fit in a sentence, because the skill already carries the rest.

Reading time: 13 minutes

Where that paragraph came from, and why it keeps growing

Nothing about this is unique to one developer at Riverside Clinics. A colleague implementing the next feature writes nearly the same paragraph from memory a week later, and gets some of it wrong: she remembers Testcontainers but forgets the problem-details format, so the new endpoint returns a different error shape than the old one. A third developer never heard about constructor injection at all, because the rule lived in a comment on a pull request he wasn't copied on, so his service uses field injection and passes review anyway, because the reviewer that day didn't catch it either.

Chapter 6 was about rules nobody had written down at all: three developers naming the same kind of object three different ways because the naming convention existed only as habit. This is a different failure. These rules usually are written down, somewhere. A wiki page. A README nobody has opened since the project started. The knowledge exists. What's missing is anywhere the AI reads it at the moment the request is made, so every request re-states it, unevenly, from whatever each developer happens to remember that day.

A request that tries to specify testing, logging, error handling, and naming all at once, in the same breath as the actual feature, is a symptom: the knowledge it's straining to carry belongs to the project, not to whichever request happens to be open in whichever chat window that afternoon. Better wording won't fix that. Even a clearer version of the same paragraph still has to be rewritten, imperfectly, every single time.

The cost shows up gradually, then all at once: a support ticket traced back to an error format that only half the endpoints follow, a new hire who copies the wrong convention because it was the example nearest to hand, a quarter spent quietly reconciling five services that all did logging differently, none of them wrong exactly, just five different people's memory of a rule that was never written where any of them, or the AI helping them, could actually find it.

What a skill is, and what it isn't

A **skill**, in Simon Martinelli's usage, is reusable implementation knowledge: how an organization writes tests, handles errors, logs state, and validates input, captured once and referenced by every feature that needs it instead of retyped into every request.

Chapter 6 drew a line between structural constraints, the ones that apply to every feature and rarely change (dependency direction, layering, naming, module boundaries), and everything beneath that line. A skill lives beneath it. Architecture says a repository belongs in the infrastructure layer. A skill names which persistence library it uses, which test harness covers it, and which convention wraps a failure before it reaches the service above it. Architecture is a constraint two teams cannot disagree on and still call themselves one codebase. A skill is a habit, and a codebase can hold the same architecture under several different, equally valid sets of habits, a claim this chapter's worked example will make concrete.

Chapter 6 also named a broader shift this book keeps returning to: prompt engineering asks how to phrase a request; context engineering asks what the assistant should already know before the request arrives. A skill is one answer to that second question, scoped to implementation conventions the way architecture answers it for structural ones. The pattern this book keeps arriving at, an interpretation earned from watching the difference play out rather than a measured finding, is that a plainly worded request backed by a complete skill produces more consistent code than a carefully worded request backed by none.

> **Architecture constrains where code goes. A skill decides what it looks like once it's there.**

Where the Model Context Protocol fits, and why a skill keeps changing

Skills, as this book describes them, are documents: written, reviewed, versioned, attached to a request instead of retyped into it. A related piece of infrastructure solves a neighboring problem by a different route. The Model Context Protocol, an open standard usually called MCP, lets an AI assistant query a live source, a coding standard, an API reference, a schema, at the moment it needs that information, rather than carrying it into every conversation by hand. The two ideas share an aim, less knowledge repeated per request, but not a mechanism: a skill is something a team writes and owns; MCP is a channel for retrieving something already written elsewhere. Which one an organization reaches for is a tooling decision. The discipline this chapter argues for, keeping implementation knowledge out of the request and in the project, holds regardless of which mechanism delivers it.

A skill is also not a document written once and shelved. When Riverside's platform team adopts a new logging library, they update the Spring Boot skill, not the memory of every developer who might ask an AI assistant to write a service next month. The next feature built against that skill inherits the change automatically, and the one after that, and the value of having written it down at all compounds with every feature built against it since.

Writing a skill instead of retyping it

1. Collect what keeps getting corrected. Look at the last several rounds of review feedback on AI-generated code for one stack, and list every correction that wasn't specific to a single feature: a testing library, a logging format, an error-wrapping convention someone keeps having to point out.
2. Separate habit from constraint. Anything the architecture already governs, layering, dependency direction, naming, module boundaries, stays in the architecture document. A skill holds only what's left: how to build within those boundaries, not where the boundaries sit.
3. Scope the skill to one stack. A Spring Boot skill and a Quarkus skill answer the same questions differently. Folding both into a single document produces a document that contradicts itself depending on which feature happens to read it.
4. State each convention as an instruction, not a principle. Not "handle errors well," but "wrap repository failures in a domain exception and return an RFC 7807 problem-details response." Specific enough that two developers implementing it independently converge on the same shape.
5. Attach the skill to the request instead of the paragraph. The request becomes a sentence naming the use case and the skill; the skill supplies everything the paragraph used to carry, and carries it the same way every time.

The Riverside Clinics booking feature, implemented against a written skill

Here is what this chapter's method produces for Riverside Clinics: a skill document the Spring Boot team keeps in the same place they keep the domain model and the architecture rules, referenced by name rather than retyped into a chat window.

Spring Boot implementation skill: Riverside Clinics booking service

Persistence. Spring Data repositories, one per aggregate root, the entity that owns a cluster of related objects (AppointmentRepository, not a generic data-access object). A repository never returns an entity straight to a controller; it's mapped to a data-transfer object first.

Dependency injection. Constructor injection only; a field-injected dependency is a review failure.

Validation. Jakarta Validation annotations on request objects (@NotNull, @Future on a requested slot time). Rules that depend on other data, like the 90-day booking window from Chapter 3, are enforced in the service layer, not the annotation layer.

Logging. Structured JSON logging through SLF4J, the Simple Logging Facade for Java. Every appointment state transition logs the appointment ID, the previous status, the new status, and the actor, patient or staff, that caused it. A patient's name or date of birth never appears in a log line.

Testing. JUnit 5. Any test that touches the database runs against Testcontainers, not an in-memory substitute. One integration test per business rule in the use case, named for the rule it checks: schedulesWithinBookingWindow_succeeds, not test1.

Error handling. A single controller-advice class translates domain exceptions into RFC 7807 problem-details responses. Repository failures are wrapped in a domain exception before they reach the service layer; nothing above persistence ever catches a raw database exception directly.

The same use case, a different skill

Send the same use case, the same domain model, the same architecture rules, to a team building the equivalent service on Quarkus instead, and only the skill changes.

> **Quarkus implementation skill: the same use case, a different stack**
>
> **Persistence.** Panache repositories rather than Spring Data interfaces: an AppointmentRepository implements PanacheRepository, and the entity carries no persistence calls of its own. The rule that a repository never hands an entity straight to a resource class still holds.
>
> **Dependency injection.** Constructor injection only, unchanged from the Spring Boot skill.
>
> **Validation.** The same Jakarta Validation annotations the Spring Boot skill requires, applied on the entity rather than a separate request object.
>
> **Logging.** Unchanged: the same structured JSON format and the same required fields as the Spring Boot skill.
>
> **Testing.** JUnit 5 with the Quarkus test extension, plus a native-image build verification step the Spring Boot skill has no equivalent for.
>
> **Error handling.** One exception-mapper class per exception type, producing the same RFC 7807 shape the Spring Boot skill produces, arrived at through a different mechanism entirely.

Both implementations satisfy the same use case, the same domain model, the same rule that a patient can't double-book a slot. Both carry out identical business logic. Only the shape of the code differs, and that shape is exactly what a skill, not a use case, decides.

Where skills go wrong

▲ **WARNING**

A common way a team reaches for consistency is one file, STANDARDS.md or something like it, that starts reasonable and never stops growing. A rule gets added for the payments feature. Then one for the reporting module. Then a security exception discovered during an audit. Nothing gets removed, because nobody's certain what's still necessary. A year or two later that file is thousands of lines long, loaded in full on every request no matter what the request is about, and it contradicts itself in at least two places nobody has caught yet.

One instruction file for everything	A reporting feature loads payment-gateway conventions it will never use, and the one rule actually relevant to it sits buried on line 640.
A skill written per feature instead of per stack	Fifteen near-identical documents, each restating the same testing convention a single Spring Boot skill should have owned once, drifting slightly further apart with each new one.

Field guide: a worked skill definition

- ☐ Persistence: which repository or entity pattern, and what is a repository never allowed to return directly to a caller?
- ☐ Dependency injection: which injection style is mandatory, and what happens at review when someone uses another one?
- ☐ Validation: which library or annotation set, and which rules belong in the service layer instead of the framework?
- ☐ Logging: what format, what must appear on every state change, and what must never appear, such as a patient's personal data?
- ☐ Testing: which framework, which harness covers a test that touches the database, and what naming convention marks which rule a test checks?
- ☐ Error handling: which exception-wrapping convention, and what shape does a client-facing error response take?

Fill in one answer per line for one stack your team actually uses. A document that leaves any line blank is a skill your team is still relying on someone's memory for.

What to remember

- A request that tries to specify testing, logging, error handling, and naming all at once is evidence that the knowledge it's carrying has nowhere else to live.
- A skill is reusable implementation knowledge, how an organization tests, logs, validates, and handles errors, kept separate from architecture's structural constraints.
- The same use case implemented against two different skills produces two different, equally correct implementations; the business logic stays fixed, and only the shape of the code carrying it out changes.
- One large instruction file feels like consistency and behaves like clutter. Several small, stack-specific skills scale in a way the giant file never does.
- A skill is a living document. Update it once, and every feature built against it afterward inherits the change without anyone retyping anything.

QUESTIONS FOR YOUR TEAM

- How many of your team's current AI prompts repeat the same testing or logging conventions, worded slightly differently each time?
- If your organization changed its error-handling convention tomorrow, how many existing prompts would need to be rewritten by hand before the new convention actually took effect?

ONE ACTION TO TAKE NEXT

Pull the last three feature requests your team sent an AI assistant for the same stack. List every implementation convention that shows up in more than one of them. That list is the first draft of your first skill.

TRANSITION. Vision, requirements, use cases, a domain model, and now architecture and skills: everything an assistant needs before it writes a line is written down. The next chapter follows all six layers through the rest of the workflow, from a finished specification to software running in production.

Skills: reusable implementation knowledge ends here.